Reflective Fold Obstruction: Internal Closure Does Not Exhaust Architectural Transition

Nova Spivack
https://www.novaspivack.com

2026

Abstract

We prove a general theorem of *fold obstruction*: if a predicate I is preserved along the primitive steps of an internal relation r, then I is preserved along the full reflexive–transitive closure r^*, and any target failing I is not internally reachable from a source satisfying I. In the hull form, when every seed in a set S_0 satisfies I and I is forward-closed for r, the entire closure hull $\mathrm{reachableFrom}(r, S_0)$ satisfies I; hence any I-violating target lies outside that hull.

We instantiate this on the *reflective slot calculus*, where sort structure (object vs. morphism) is preserved by disciplined internal reflective steps, blocking any internal route from the object slot to the morphism representer. A bridge theorem shows that every upstream *representational system* inherits the reflective fold obstruction. Additional dynamic instances—including holonomy-phase and open–compact witness obstructions with nontrivial step relations—are formalized in the artifact.

Semantic type obstruction lifts the same hull mechanism to a preorder on *tags*: provably, **Turing-completeness does not imply semantic-type completeness**. A system may be maximally powerful in the standard computational sense and yet be permanently bounded in which architectural modes (*types of being*) it can *instantiate* along its *own* primitive dynamics—distinct from what it can *simulate*. We prove that *typed simulations* pull specification-level type gaps back to simulating systems, while a stricter *realization* witness (a section with backward step-lifting) is exactly the extra structure needed for a machine-checked *bidirectional* agreement of type reachability between a model and a pulled-back typing.

The entire development is machine-checked in Lean 4/Mathlib with zero `sorry` and zero `admit` across all shipped modules.

Trust boundary: Fold and hull theorems are internal to the Lean relation calculus formalized here; mapping to physical or institutional "architectures" is an external modeling task.

Archival version (Zenodo): [DOI pending publication]

Keywords: reflective fold, internal closure, invariant hull, Lean 4, semantic types
MSC 2020: 03F40, 03B70, 68T27

Code/Data availability:
`https://github.com/novaspivack/reflective-fold-obstruction-lean`
(reflective-fold-obstruction development; `lakebuild`; see repository `README` and `lakefile.lean`.)

1 Premises, disagreement coordinates, and trust boundary

Where disagreement can (and cannot) live

Formal spine. Obstruction lemmas are tied to `ReflTransGen` reachability and named preservation predicates in the artifact.

Premise coordinates (for external readers).

- **Primitive-step choice.** Conclusions depend on the declared step relation r and its reflexive–transitive closure; changing r changes the hull. *Logical effect:* Industrial workflow graphs are not automatically isomorphic to the formal r without a verified functor bridge.
- **Semantic-type strand.** Turing-completeness claims concern standard computability predicates alongside tag preorders formalized in Lean; they do not subsume informal philosophy of mind without translation. *Logical effect:* Accepting the computational lemma leaves metaphysical packaging optional.

2 The problem: internal closure versus architectural transition

Consider a system whose states evolve along an *internal* step relation $r : \alpha \to \alpha \to \text{Prop}$. Reflexive–transitive closure r^* captures everything the system can reach by iterating, composing, and idling along r. A natural question is:

> *Does internal closure exhaust architectural transition? That is, can every state reachable "in principle" be obtained by iterating r?*

This paper answers: **no**, in a precise and general sense.

We prove that whenever a predicate I is preserved by each primitive r-step, the predicate lifts to the entire closure r^*. Any target violating I is therefore *not* internally reachable from a source satisfying I. We call such a blocked transition a *fold*: a structural passage that the internal calculus cannot generate.

A fold is not an iterate

The central claim of this paper: any architecture generated solely by internal reachability remains confined to its invariant class. Mismatch across that boundary certifies that the target requires a *fold*—a passage not generated by r^*. Internal closure and architecture-changing transition are structurally different operations.

The claim is not that the unreachable target is impossible in all senses, but that any route generated only by r and preserving I cannot reach it. Internal closure and architecture-changing transition must therefore be distinguished.

A second orthogonal ceiling (preview). Classical computability answers how much a Turing-complete model can *compute*; this paper's fold machinery answers how far a fixed architecture can move *within itself* under a chosen primitive dynamics. These are independent: internal iteration may preserve invariants that cap *semantic types*—modes of self-relation or

adjudication the system *instantiates*—even when no computability-theoretic barrier blocks *simulating* deeper behaviour externally. Section 10 makes this split precise at the level of machine-checked *type reachability* and simulation, and Section 12 summarizes the stakes.

Computational vs. architectural ceilings

Do not conflate them. Turing-completeness settles how much a standard model can *compute externally*; fold obstruction settles how far a fixed system can move *inside its own* primitive dynamics while preserving chosen invariants. The semantic-type layer (Section 10) proves the second can bound *kinds of being* (tags) even when the first is maximal—**instantiation along r is not simulation in general**.

3 Definitions

We work over a type α in a universe u. All definitions and theorems in this section have exact counterparts in the Lean 4 artifact.

Definition 3.1 (Primitive internal relation)**.** A *primitive internal relation* on α is a binary relation $r : \alpha \to \alpha \to \mathrm{Prop}$. We make no assumptions on r (it need not be reflexive, transitive, or decidable).

Definition 3.2 (Forward closure / preserved invariant)**.** A predicate $I : \alpha \to \mathrm{Prop}$ is *forward-closed* for r (equivalently, *preserved by* r) when

$$\mathrm{ForwardClosed}(r, I) \;:=\; \forall\, a\, b,\;\; r(a,b) \implies I(a) \implies I(b).$$

We write $\mathrm{PreservedBy}(r, I)$ as a synonym.

Definition 3.3 (Reflexive–transitive closure)**.** $\mathrm{ReflTransGen}(r)$ is the reflexive–transitive closure of r, the standard Mathlib [4] `Relation.ReflTransGen`:

- $\mathrm{ReflTransGen}(r, a, a)$ for every a (*refl*);
- if $\mathrm{ReflTransGen}(r, a, b)$ and $r(b, c)$, then $\mathrm{ReflTransGen}(r, a, c)$ (*tail*).

Definition 3.4 (Closure hull)**.** Given a seed set $S_0 \subseteq \alpha$, the *closure hull* is

$$\mathrm{reachableFrom}(r, S_0) \;=\; \big\{\, y \in \alpha \;\big|\; \exists\, x \in S_0,\; \mathrm{ReflTransGen}(r, x, y) \,\big\}.$$

4 The main theorem

The theorem is completely general in the carrier type α and primitive relation r. No categorical, topological, or computational assumptions are required beyond preservedness of the chosen invariant I.

Theorem A: primitive-to-closure preservation

Theorem 4.1 (Closure preservation)**.** *Let* $r : \alpha \to \alpha \to \mathrm{Prop}$ *and* $I : \alpha \to \mathrm{Prop}$ *with* $\mathrm{ForwardClosed}(r, I)$*. Then for all* $a, b : \alpha$*,*

$$\mathrm{ReflTransGen}(r, a, b) \implies I(a) \implies I(b).$$

Proof. By induction on the derivation of $\mathrm{ReflTransGen}(r, a, b)$. The *refl* case is immediate. For the *tail* case, given $\mathrm{ReflTransGen}(r, a, b')$ and $r(b', b)$, the inductive hypothesis yields $I(b')$, and forward closure gives $I(b)$. □

Lean: `internal_reachability_preserves_invariant`.

Theorem B: pairwise fold obstruction

Theorem 4.2 (Fold obstruction)**.** *If* $\mathrm{PreservedBy}(r, I)$*,* $I(S)$*, and* $\neg I(T)$*, then*

$$\neg\ \mathrm{ReflTransGen}(r, S, T).$$

Proof. Suppose for contradiction that $\mathrm{ReflTransGen}(r, S, T)$. By Theorem 4.1, $I(S)$ implies $I(T)$, contradicting $\neg I(T)$. □

Lean: `fold_obstruction_of_invariant_mismatch`.

The master obstruction

Any architecture generated solely by internal reachability remains confined to its invariant class; mismatch across that boundary certifies that the target requires a fold. The obstacle is structural: no amount of iteration, composition, or closure under r can cross the boundary drawn by a preserved I.

Remark 4.3. Theorem 4.2 does not say that T is "impossible." It says that any path generated exclusively by r from a source satisfying I cannot reach a target violating I. Hence internal closure and invariant-breaking transition must be distinguished: the latter requires a *fold*—a structural passage not in r^*.

Theorem C: hull exclusion

Theorem 4.4 (Hull preservation)**.** *If* $\mathrm{PreservedBy}(r, I)$ *and* $\forall s \in S_0,\ I(s)$*, then*

$$\forall\, x \in \mathrm{reachableFrom}(r, S_0),\ I(x).$$

Proof. Let $x \in \mathrm{reachableFrom}(r, S_0)$, so there exists $s \in S_0$ with $\mathrm{ReflTransGen}(r, s, x)$. Since $I(s)$ holds and I is forward-closed, Theorem 4.1 gives $I(x)$. □

Corollary 4.5 (Hull exclusion)**.** *Under the hypotheses of Theorem 4.4, if* $\neg I(T)$*, then* $T \notin \mathrm{reachableFrom}(r, S_0)$*.*

Lean: `reachableFrom_hull_preserves_invariant`, `not_mem_reachableFrom_of_preserved_invariant_mismatch`.

5 Reflective instantiation: sort obstruction

The abstract theorem is not vacuous. We instantiate it on a canonical reflective architecture, demonstrating a genuine dynamical obstruction at the *sort* level.

Definition 5.1 (Reflective system)**.** A *reflective system* R consists of a category $\mathcal{C}$, a distinguished object $A \in \mathcal{C}$, and a self-representation endomorphism $\mathsf{represent} : A \to A$. This is the minimal categorical setting in which "iterate the self-representation internally" becomes mathematically meaningful: the endomorphism monoid $\mathrm{End}(A)$ supplies the iteration, and the category supplies the ambient structure [1, 2].

Definition 5.2 (Ontological slot)**.** The *ontological slot* type on R is the disjoint union

$$\mathrm{Slot}(R) \;=\; \mathsf{obj}(\mathcal{C}) \;+\; \mathsf{mor}(\mathrm{End}(A)),$$

tagging each element as either an *object tag* $\mathsf{obj}(c)$ for $c \in \mathcal{C}$, or a *morphism tag* $\mathsf{mor}(f)$ for $f : A \to A$. These constructors are injective and disjoint: $\mathsf{obj}(c) \neq \mathsf{mor}(f)$ for all c, f.

Definition 5.3 (Reflective slot step)**.** The primitive internal relation $\mathsf{reflectiveSlotStep}(R)$ on $\mathrm{Slot}(R)$ is defined by:

- $s = t$ (*idle*); or
- $s = \mathsf{mor}(f)$, $t = \mathsf{mor}(g)$, and $g = f \circ \mathsf{represent}$ (*mor advance*).

Object slots cannot be stepped into morphism slots: the only moves from $\mathsf{obj}(c)$ are reflexive.

Definition 5.4 (Object-branch predicate)**.** $\mathrm{IsObj}(s) \coloneqq \exists\, c,\ s = \mathsf{obj}(c)$.

Lemma 5.5 (Sort preservation)**.** $\mathrm{ForwardClosed}(\mathsf{reflectiveSlotStep}(R), \mathrm{IsObj})$.

Proof. If s is an object tag $\mathsf{obj}(c)$ and $\mathsf{reflectiveSlotStep}(R, s, t)$, then either $t = s$ (idle), or s would need to be a morphism tag (impossible for $\mathsf{obj}(c)$). Hence t is still an object tag. □

Lemma 5.6. $\neg\,\mathrm{IsObj}\big(\mathsf{mor}(\mathsf{represent})\big)$.

Proof. If $\mathsf{mor}(\mathsf{represent}) = \mathsf{obj}(c)$ for some c, this contradicts the constructor disjointness of the slot type. □

Theorem 5.7 (Reflective fold obstruction — sort mismatch)**.** *For any reflective system R,*

$$\neg\ \mathrm{ReflTransGen}\big(\mathsf{reflectiveSlotStep}(R),\ \mathsf{obj}(A),\ \mathsf{mor}(\mathsf{represent})\big).$$

Proof. By Theorem 4.2 with $I \coloneqq \mathrm{IsObj}$, using Theorem 5.5 (preservation), $\langle A, \mathrm{rfl}\rangle$ (source satisfies I), and Theorem 5.6 (target violates I). □

Lean: `ReflectiveFold.reflective_fold_obstruction_slot_mismatch`.

Sort structure is a fold boundary

In a reflective system, the passage from the object slot $\mathsf{obj}(A)$ to the morphism representer $\mathsf{mor}(\mathsf{represent})$ is **not an iterate**: no chain of internal reflective steps can cross the sort boundary. The self-representation endomorphism generates unbounded depth within the morphism branch, but it cannot bridge the structural gap between object and morphism. This is the precise sense in which **a fold is not an iterate**.

Remark 5.8 (Rich reflective calculus)*.* The obstruction survives under the *rich reflective calculus* $\mathsf{reflectiveCalcStep}(R)$, which extends the primitive step by allowing mor-branch jumps $f \mapsto f \circ \mathsf{represent}^n$ for *any* positive iterate $n \geq 1$. Sort preservation holds by the same argument: object tags admit no morphism-branch transitions. Under the hypothesis $\mathrm{IterInjective}(R)$ (distinct natural-number exponents give distinct endomorphism powers), the rich calculus is *strictly* stronger than the primitive slot step.

6 A dynamic instance: holonomy-phase obstruction

The reflective slot obstruction of Section 5 is the primary instance. To show the abstract theorem is not limited to sort-level reasoning, we highlight one further instance with genuinely nontrivial dynamics.

Example 6.1 (Holonomy-phase obstruction)**.** The carrier is HolonomyState, a gated phase counter. The internal relation holonomyPhaseStep advances the phase under a nontrivial gating condition—it is *not* mere equality. The invariant HolonomyTrivialBounded (trivial holonomy tag with bounded phase) is forward-closed for this step. A target state with twist holonomy lies outside the closure hull of trivially-tagged seeds. Structurally, this means that a system whose holonomy phase evolves under the gated dynamics *cannot* reach a twisted configuration from a trivial one by internal iteration alone: the twist requires a fold.

The machine-checked artifact formalizes additional instances: boundary-type, connected-boundary, orientability-parity, holonomy-tag, and open–compact witness obstructions. Each is a one-line corollary of Theorem 4.2 with its own carrier, invariant, and step relation. These are catalogued in the artifact's theorem inventory.

7 Relation to representational incompleteness

Reflective fold obstruction (RFO) and representational incompleteness (RI) [6] are complementary programs.

RI proves that no finitely-typed system can universally parametrize its own function space: the diagonal / Lawvere barrier [1] blocks total self-capture. The obstacle is a *cardinality*-flavored impossibility (surjectivity failure of curry).

RFO proves that certain architectural transitions—those that would violate a preserved invariant—cannot be realized by internal closure. The obstacle is a *transport*-flavored impossibility (invariant mismatch along reachability).

Atop that dynamical core, Section 10 packages a *semantic typing* layer: tags, instantiation, and type reachability. It answers a question orthogonal to RI's diagonal barrier—how internal dynamics bounds *architectural kinds*—while still machine-checkable without importing a full Rice-style index-set formalization.

Diagonal pressure vs. dynamical hull vs. semantic types

ri: no universal self-parametrization of the function space (Lawvere/diagonal). **rfo**: preserved invariants close internal hulls (fold, not iterate). **Semantic types (Section 10)**: the same hull story at the tag layer, including provable **limits on Turing-complete hosts** that are not the halting or Rice plotlines. Keep the flavours separate: they reinforce each other, but none reduces to the others.

The two programs share reflective-system infrastructure but own different theorems. Every upstream RI host inherits the RFO sort obstruction through a proved bridge:

Theorem 7.1 (RI bridge)**.** *For any* RepresentationalSystem R *(in the sense of* RI*), the induced* ReflectiveSystem *satisfies*

$$\neg\ \mathrm{ReflTransGen}\big(\mathsf{reflectiveSlotStep},\ \mathsf{obj}(A),\ \mathsf{mor}(\mathsf{represent})\big).$$

Lean: `representational_incompleteness_implies_reflective_fold_obstruction`.

Remark 7.2. The bridge adds *no new axioms*: it is definitional alignment plus the generic reflective slot theorem. The hypothesis IterInjective is required only for the tower-unboundedness channel (a separate, complementary obstruction), not for the fold statement.

8 Robustness of the obstruction

The fold pattern is not brittle. Three meta-stability results ensure that the obstruction survives routine extensions of the primitive relation.

Proposition 8.1 (Subrelation persistence)**.** *If $r \subseteq r'$ as graphs and* $\mathrm{ForwardClosed}(r', I)$*, then* $\mathrm{ForwardClosed}(r, I)$*. The fold obstruction for r follows from preservation on a single larger relation r', without a second induction.*

Proposition 8.2 (Generator-wise preservation)**.** *Given an indexed family $\{r_i\}_{i \in \iota}$, define the generated one-step relation* $\mathrm{unionGen}(\{r_i\})(x, y) \coloneqq \exists\, i,\ r_i(x, y)$*. If* $\mathrm{ForwardClosed}(r_i, I)$ *for every i, then* $\mathrm{ForwardClosed}(\mathrm{unionGen}(\{r_i\}), I)$*, and the fold obstruction holds for the generated calculus.*

Additionally, for any indexed family of admissible reflective generators, each preserving sort separation, the union relation inherits the fold obstruction. The single-step rich calculus and its ReflTransGen-hull are shown to lie inside the canonical generator-union hull, so the generated obstruction subsumes the hand-written one.

The obstruction is stable

Strengthening the internal calculus—adding new primitive steps, taking unions of generator families, or passing to richer reflective dynamics—does not escape the obstruction. As long as the enlarged relation still preserves the invariant, the fold boundary holds. This makes the theorem a genuine *obstruction calculus*, not a fragile artifact of a specific step system.

9 Significance

Theorems 4.2 and 5.7 together establish a clean boundary between what internal closure can generate and what it cannot.

(1) **The theorem separates internal closure from architecture-changing transition.** Any route generated only by the chosen internal relation r, from a source satisfying I, cannot reach a target violating I. Passages that cross an invariant boundary require a *fold*—a transition not generated by r.

(2) **The reflective instance shows the theorem is nonvacuous in a canonical self-referential setting.** Sort structure is preserved under disciplined internal reflective dynamics, so the object-to-representer passage is genuinely blocked.

(3) **The artifact proves this is a machine-checked obstruction calculus, not an informal schema.** The entire development, from abstract definitions through concrete instances, compiles with zero proof holes.

(4) **Semantic types vs. Turing completeness (Section 10).** The library proves that the preorder of *semantic-type reachability* is genuinely nontrivial and that *internal* type gaps persist regardless of how much "raw computation" the host could in principle support: simulation of a deeper configuration is not instantiation of its type along the host's primitive step. Typed simulations preserve forward reachability and pull *obstructions* back; a proved *section* criterion identifies when this becomes an *iff*—the formal edge between mere simulation and *realization* at the typing layer.

Novelty

We do not claim that preserved-invariant reasoning or reflexive–transitive closure [3, 4] are individually new. A central novelty of this paper is the formal realization of the principle that *a fold is not an iterate*: internally generated reachability, even under closure and enriched calculi, remains confined to invariant-preserving classes, so invariant-breaking architectural transitions cannot be obtained by iteration alone. The contribution is the **machine-checked integration** in which this principle is proved, instantiated on reflective architectures, extended to generated calculi and admissible families, and bridged to upstream representational incompleteness—all with zero proof holes.

The novelty is the formal realization

Individual ingredients are standard. What is new is a machine-checked integration in which preserved invariants lift from primitive steps to reflexive–transitive closure, yielding pairwise obstruction, hull exclusion, reflective sort obstruction, generated-calculus stability, and bridge compatibility in one theorem stack—so that internal iteration alone is provably insufficient for invariant-breaking architectural transition. The same stack now includes a *semantic-type* layer with explicit theorems on pullbacks, simulation, complementary-tag gaps under one-sided preservation, and seed-local hull exclusion—articulating limits on Turing-complete hosts that are not the classical halting/Rice storyline, but dynamical obstruction at the level of what a system *is* under its own transition relation.

10 Semantic type obstruction

Turing completeness and an orthogonal semantic ceiling

Classical computability fixes a system's *computational* ceiling: under standard notions of algorithm, a Turing-complete model can compute every partial recursive function and no others. That theorem classifies *external* functional behaviour over standard presentations of algorithms. It does *not* classify what *semantic types* a system can *instantiate*—modes of self-relation, self-modeling, or adjudication—as matters of architectural *being* along its *own* internal transition relation.

The machine-checked layer below shows these dimensions are **orthogonal**. A host may be Turing-complete (hence able to simulate rich counterfactual dynamics) while its primitive calculus preserves predicates that make certain tag pairs *internally* unreachable forever: no sequence of admissible internal steps joins a source tag to a target tag, even though the host could *simulate* a machine that appears to realize the target tag externally. What is proved

is not a replacement for Rice or the halting problem, but a *different* impossibility template: **invariant preservation along r closes the hull**, at the state level and then at the tag level.

Turing-complete $\neq$ semantic-type complete

Orthogonal limits. A host may run every partial recursive function in the usual sense and still be *permanently type-bounded*: its primitive step can preserve predicates that make some tag pairs unreachable by any internal chain. **Simulation does not repair the gap**: the host may encode a counterfactual machine that "would" change type, without itself *instantiating* that transition natively. What is machine-checked is dynamical obstruction—**hulls closed by preserved invariants**—not a substitute for Rice or the halting problem.

Semantic typing

Fix a carrier of *systems*, a discrete set of *tags* (semantic types), a binary *primitive step* relation, and a predicate stating when a system *instantiates* a tag. If every primitive step preserves each tag's extension, then any reflexive–transitive internal route from a system instantiating tag T stays inside that tag's invariant class. Thus two tags separated by an invariant mismatch are not connected by internal iteration; this is the same hull mechanism as Theorem 4.2, lifted to a preorder on tags.

Tags inherit the fold boundary

Semantic tagging is not a separate metaphysics: it is Theorem 4.2 with predicates indexed by tags. **Preserved tag extensions** mean that every internal route stays inside the invariant class of each tag it touches; **type gaps** are mappings incompatible with that preservation. The move from states to tags packages the same obstruction for comparing *architectural kinds*.

Non-triviality

The *type reachability* preorder on tags is not total: concrete witnesses exhibit pairs (T, T') with no connecting internal chain. One canonical family uses *self-modeling depth* labels on $\mathbb{N}$ with identity dynamics: depth is preserved, so tag `depth(0)` cannot reach `depth(1)`—a crisp toy model of "type-bounded" introspection. A second family abstracts *adjudication class* (total-effective vs. weaker behaviour): kind is preserved along steps, so the total tag cannot reach the non-total tag without a fold. A third instance reframes the reflective *sort* gap: the object-branch tag cannot reach the morphism-carrier tag under `reflectiveSlotStep`, so the diagonal witness `mor(represent)` is unreachable from the parametric object branch—in parallel with the RI bridge (Section 7).

The type preorder is provably nontrivial

Not all tags are comparable by internal iteration. The artifact gives concrete witness families: self-modeling *depth* on $\mathbb{N}$ (identity dynamics), abstract *adjudication class* under preserved kind, and the reflective *sort* gap (object branch vs. morphism carrier) aligned with the RI bridge. Together they nail down that **semantic-type reachability is genuinely nontrivial as a preorder**, not a vacuous afterthought to Turing completeness.

Simulation, pullback, and realization

To compare architectures, the library formalizes *typed primitive simulations*: a map π from a domain state space to states of a fixed typing, carrying the domain's primitive relation into the typing's primitive steps. Pulling tags back along π yields a typing on the domain (`pullbackAlongSimulation`). **Forward simulation** is always available: type reachability in the pullback implies type reachability in the codomain (via a homomorphism lift of `ReflTransGen`).

The converse is conditional—and this is the formal simulation/realization split. In general one cannot expect backward reachability in the pullback from the codomain alone: simulation need not exhibit all trajectories of the simulated system in the host's native dynamics. The artifact isolates a minimal witness that repairs the gap: a *section* σ with $\pi \circ \sigma = \mathrm{id}$ that *lifts every primitive step backward* (`TypedPrimitiveSimulationSection`). Under exactly this data, **type reachability is equivalent** between the codomain typing and the pullback, and the same holds for packaged `SemanticTypeObstruction` (`typeReachable_pullback_iff_of_section`). Thus "realizing" the foreign dynamics inside the host's step relation is a strictly stronger claim than forward simulation, and the library states the strengthening in provable `iff` form rather than by axiom.

Simulation vs. realization (machine-checked split)

Forward half is free. Typed primitive simulations always *project* type reachability to the codomain; type gaps on the specification *pull back* to any simulating dynamics. **The converse costs structure.** Without backward step-lifting, the host may simulate without realizing every codomain trajectory. A section σ with $\pi \circ \sigma = \mathrm{id}$ that lifts each primitive step backward (`TypedPrimitiveSimulationSection`) is the minimal repair: then **type reachability matches the pullback iff it matches the model**, and the same for packaged obstruction (`typeReachable_pullback_iff_of_section`). **Realization = bidirectional agreement**, not "bigger state space."

Complementary tags and seed-local hulls (weaker hypotheses)

Many applications single out one invariant P whose forward-closure is cheap, without assuming a fully bipartite typing where $\neg P$ is also step-closed. Two results capture this cleanly: **complementary-tag gaps** (`typeGap_of_complementary_tags`) propagate from two tags that classify P and $\neg P$ respectively, requiring only that P itself be forward-closed along primitive steps; and **seed-local hull exclusion** (`false_tag_not_mem_tagsOnHull_of_preserved_seed`)

shows that a tag for $\neg P$ never appears on the primitive hull from P-seeds under the same one-sided hypothesis. These lemmas are obstruction-theoretic footnotes to the master hull theorem, but they match how practitioners reason about reachable components from real initial conditions.

One-sided preservation is often enough

Many arguments fix a predicate P whose forward-closure is cheap, without assuming a full bipartite discipline in which $\neg P$ is also step-closed. **Complementary-tag gaps** (`typeGap_of_complementary_tags`) need only that P be preserved along primitive steps, with two tags classifying P and $\neg P$. **Seed-local hull exclusion** (`false_tag_not_mem_tagsOnHull_of_preserved_seed`) shows a $\neg P$-tag never appears on the hull from P-seeds under the same hypothesis. These lemmas target the *reachable component from realistic initial conditions*, not a global state space.

Further corollaries and scope

The library also includes: *primitive extension* monotonicity; *tags-on-hull* from seeds; *pair-tag* conjunction obstruction; a *bipartition* bridge to `FoldObstructionBundle` when both P and $\neg P$ are forward-closed; an infinite *pairwise antichain* of depth labels; and an explicit computability *scope note* contrasting Rice-style index phenomena with this *dynamical* formulation (no smuggled formalization of r.e. degrees is required to use the typing theorems).

Lean anchors

The development lives under
`ReflectiveFoldObstruction/SemanticType/`
with stable import surface `Core/SemanticTypeObstruction`.

Core.

- `semanticType_preorder_nontrivial`
- `selfModelDepth_obstruction`
- `adjudication_semantic_obstruction`
- `RI_semantic_type_mismatch`

Simulation / realization.

- `typeGap_simulation_pullback`
- `typeReachable_pullback_iff_of_section`
- `SemanticTypeObstruction_pullback_iff_of_section`

Complementary / hull.

- `typeGap_of_complementary_tags`
- `false_tag_not_mem_tagsOnHull_of_preserved_seed`

See also Table 1.

11 Machine-checked artifact

Component	Value
Proof assistant	Lean 4
Toolchain	`leanprover/lean4:v4.29.0-rc6`
Library	Mathlib v4.29.0-rc6
Build command	`lake build ReflectiveFoldObstruction`
`sorry` count	0
`admit` count	0

Theorem anchors

Table 1 lists the principal Lean names.

Role	Lean name
Closure preservation (Theorem 4.1)	`internal_reachability_preserves_invariant`
Fold obstruction (Theorem 4.2)	`fold_obstruction_of_invariant_mismatch`
Hull preservation (Theorem 4.4)	`reachableFrom_hull_preserves_invariant`
Hull exclusion (Theorem 4.5)	`not_mem_reachableFrom_of_preserved_invariant_mismatch`
Sort preservation (Theorem 5.5)	`reflective_step_preserves_sort_separation`
Reflective fold (Theorem 5.7)	`ReflectiveFold.reflective_fold_obstruction_slot_mismatch`
Rich calculus fold (Theorem 5.8)	`reflectiveCalc_fold_obstruction_slot_mismatch`
RI bridge (Theorem 7.1)	`representational_incompleteness_implies_reflective_fold_obstruction`
Subrelation persistence (Theorem 8.1)	`preserved_under_relation_extension`
Generated-calculus fold (Theorem 8.2)	`generated_calculus_fold_obstruction`
Semantic-type non-totality (Section 10)	`semanticType_preorder_nontrivial`
RI semantic mismatch (Section 10)	`RI_semantic_type_mismatch`
Relation map / simulation (Section 10)	`reflTransGen_map`, `typeGap_simulation_pullback`, `typeReachable_pullback_iff_of_section`, `typeGap_of_complementary_tags`, `false_tag_not_mem_tagsOnHull_of_preserved_seed`

Table 1: Principal machine-checked theorem anchors.

Reproduction

```
cd reflective-fold-obstruction-lean
lake update && lake exe cache get
lake build ReflectiveFoldObstruction
```

The full module tree and formalization map are given in Section A.

12 Conclusion

This paper's core contribution is a general, machine-checked *fold obstruction*: preserved invariants propagate along reflexive–transitive closure, so any target that violates the invariant lies outside internal reachability from compliant sources. The principle is instantiated on reflective sort dynamics, extended to rich calculi and generator unions, and bridged to representational incompleteness without new axioms.

The *semantic type* layer (Section 10) pushes the same idea to tags and makes vivid a limit that is easy to conflate with classical computability but is logically distinct: **Turing-completeness does not entail semantic-type completeness**. A system may compute everything a textbook universal machine computes, while its native step relation preserves architectural predicates that *permanently* exclude certain semantic kinds from the internally reachable class. Simulation can mask this from outside observers; what the formalization rewards is careful separation between *what can be forward-simulated* and *what is realized along the host's own primitive transitions*—with a proved `iff` characterizing the latter in terms of backward-lifting sections.

Together, the development supports a unified moral for reflective architectures and their machine-checked obstructions: **internal closure is not architectural universality**. Iteration along a disciplined dynamics respects invariants; crossing an invariant boundary is a fold, not an iterate—whether the invariant is sort structure, holonomy data, boundary connectivity, semantic adjudication class, or the instantiation of a finer semantic tag.

Closing moral

Internal closure is not architectural universality. Iteration along a chosen primitive dynamics respects preserved invariants; crossing an invariant boundary is a *fold*. Classical computability fixes the *computational* ceiling; this development fixes a complementary *semantic-dynamical* ceiling—including a proved distinction between what can be *simulated forward* and what is *realized* with backward-lifting structure. **Three orthogonal stories** work together: RFO folds, RI diagonal pressure, and semantic-type obstruction along r—each machine-checked, each with its own impossibility flavour.

References

[1] F. W. Lawvere. Diagonal arguments and cartesian closed categories. In *Category Theory, Homology Theory and Their Applications II*, Lecture Notes in Mathematics, vol. 92, pp. 134–145. Springer, 1969.

[2] N. S. Yanofsky. A universal approach to self-referential paradoxes, incompleteness and fixed points. *Bulletin of Symbolic Logic*, 9(3):362–386, 2003.

[3] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*. Studies in Logic and the Foundations of Mathematics, vol. 103. North-Holland, revised edition, 1984.

[4] The Mathlib Community. *Mathlib*: the Lean mathematical library. `https://github.com/leanprover-community/mathlib4`, 2020–2026.

[5] L. de Moura and S. Ullrich. The *Lean 4* theorem prover and programming language. In *CADE-28*, Lecture Notes in Computer Science, vol. 12699, pp. 625–635. Springer, 2021.

[6] N. Spivack. Representational incompleteness: no self-modeling system can capture its own diagonal. Preprint, 2026. Machine-checked in Lean 4/Mathlib.

A Module map

Module path	**Role**
`Core/Basic`	`ReflectiveSystem`, `IterInjective`, iterates, monoid laws
`Core/Slots`	`OntologicalSlot`, sort disjointness, `ReflectiveSlot`
`Core/ArchitectureObstruction`	Stable re-export names for fold pattern
`Core/SemanticTypeObstruction`	`SPEC_020`: semantic typing + three instances + corollaries
`SemanticType/*`	Core bundle; depth / adjudication / RI diagonal; hull tags (seed-local exclusion); pair tags; bipartition bridge; simulation pullback + section (*realization*) `iff` on reachability; complementary tags; antichain; computability scope note
`Reachability/InternalOps`	`ForwardClosed`, `PreservedBy`, monotone `ReflTransGen`, relation extension
`Reachability/ClosureHull`	`reachableFrom`, hull algebra (union, intersection, idempotence), subrelation monotonicity
`Reachability/Invariants`	Hull–invariant wrappers
`Reachability/ReflectiveSteps`	`reflectiveSlotStep`, sort preservation
`Reachability/ReflectiveCalculus`	Rich calculus, admissible unions, sub-hull containment
`Reachability/GeneratedCalculi`	`unionGen`, reflective Bool generators
`Reachability/ArchitectureClasses`	`Architecture`, `FoldObstructionBundle`
`Obstruction/Fold`	Summit: fold obstruction, hull exclusion, relation-extension persistence
`Obstruction/CanonicalInstances`	Boundary, connectivity, parity, holonomy, sort corollaries
`Obstruction/ReflectiveFold`	Sort mismatch flagship, tower unboundedness
`Obstruction/ArchitectureUniversality`	Bundled fold + hull theorems
`Obstruction/OpenCompactWitness`	Discrete witness-step obstruction on $\mathbb{N}$
`Invariants/*`	Sort separation, transport, boundary type, connectivity, orientability-like, homeomorph transport
`Topology/*`	Models, local 1D/2D, holonomy tags, holonomy phase, Hausdorff
`Examples/Representational-Incompleteness`	RI bridge: `toReflectiveSystem`, sort fold
`Examples/CylinderMobius`	Holonomy parity instance
`Examples/ObserverBridge`	Schematic RFO $\to$ OE route

All modules compile with zero `sorry` under `lake build ReflectiveFoldObstruction`.